
TRÍ TUỆ NHÂN TẠO

Machine Learning

Phần 9 — Deep Reinforcement Learning

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	4
1 Học tăng cường sâu (Deep Reinforcement Learning)	5
1.1 Học tăng cường sâu là gì?	5
1.2 Các khái niệm then chốt	5
1.3 Cách hoạt động của Deep RL	6
1.4 Danh sách thuật toán Deep RL	7
1.5 Ứng dụng của Deep RL	7
1.5.1 Trò chơi (Gaming)	7
1.5.2 Điều khiển robot (Robot Control)	7
1.5.3 Xe tự lái (Self-driving Cars)	8
1.5.4 Y tế (Healthcare)	8
1.6 So sánh RL và Deep RL	8
2 Thuật toán học tăng cường sâu (Deep RL Algorithms)	9
2.1 Học tăng cường (Reinforcement Learning)	9
2.2 Vai trò của học sâu trong RL	9
2.3 Lợi ích kết hợp học sâu và RL	9
2.4 Các thuật toán Deep RL phổ biến	10
2.4.1 Deep Q-Networks (DQN)	10
2.4.2 Double Deep Q-Networks (DDQN)	10
2.4.3 Dueling Deep Q-Networks (Dueling DQN)	11
2.4.4 Policy Gradient Methods	11
2.4.5 Proximal Policy Optimization (PPO)	11
3 Mạng Q sâu (Deep Q-Networks)	12
3.1 Deep Q-Network là gì?	12
3.2 Các thành phần chính	12
3.3 Cách hoạt động của DQN	13
3.3.1 Kiến trúc mạng nơ-ron	14
3.3.2 Experience Replay	14
3.3.3 Target Network	14
3.3.4 Chính sách Epsilon-Greedy	14
3.3.5 Quy trình huấn luyện	14
3.4 Hạn chế của Deep Q-Networks	15
3.5 Double Deep Q-Networks	15
3.6 Dueling Deep Q-Networks	16
4 Deep Deterministic Policy Gradient (DDPG)	16
4.1 Khái niệm then chốt	17
4.2 Thành phần cốt lõi (actor-critic)	17
4.3 Cách DDPG hoạt động	17
4.4 Mạng target và soft update	18
4.5 Khám phá-khai thác	19
4.6 Thách thức	19

5	Phương pháp vùng tin cậy (Trust Region)	20
5.1	Định nghĩa vùng tin cậy	20
5.2	Vai trò trong tối ưu chính sách	20
5.3	TRPO — Trust Region Policy Optimization	21
5.4	PPO — Proximal Policy Optimization	21
5.5	Natural Gradient Descent	22
5.6	So sánh nhanh	22
5.7	Thách thức	23

Đặng Tấn Quý

Lời nói đầu

Nguồn

Tài liệu biên soạn và dịch đầy đủ từ Tutorialspoint Machine Learning — mục Deep Reinforcement Learning: https://www.tutorialspoint.com/machine_learning/deep_reinforcement_learning.htm.

Cách dùng

- Đọc sau phần Reinforcement Learning.
- Chạy code từ thư mục 9.DeepReinforcementLearning/.

Đặng Tấn Quý

1 Học tăng cường sâu (Deep Reinforcement Learning)

1.1 Học tăng cường sâu là gì?

Deep Reinforcement Learning (Deep RL)

Học tăng cường sâu là nhánh con của học máy, kết hợp **học tăng cường** với **học sâu**. Deep RL giải quyết bài toán cho phép agent tính toán học ra quyết định bằng cách tích hợp học sâu từ dữ liệu đầu vào **không có cấu trúc**, mà **không cần** kỹ thuật thủ công để thiết kế không gian trạng thái. Các thuật toán Deep RL có khả năng quyết định hành động nào nên thực hiện để tối ưu mục tiêu ngay cả khi đầu vào có kích thước lớn.

1.2 Các khái niệm then chốt

Khối xây dựng Deep RL

Các khối xây dựng của học tăng cường sâu gồm mọi khía cạnh hỗ trợ học và agent ra quyết định. Môi trường hiệu quả được tạo ra nhờ sự phối hợp của các yếu tố sau.

Agent (Tác tử)

Agent là thực thể **học** và **ra quyết định**, tương tác với môi trường. Agent hành động theo **chính sách** (policy) và tích lũy **kinh nghiệm**.

Environment (Môi trường)

Môi trường là hệ thống bên ngoài agent mà agent giao tiếp. Môi trường phản hồi cho agent dưới dạng **phần thưởng** hoặc **hình phạt** tùy theo hành động của agent.

State (Trạng thái)

Trạng thái biểu diễn tình huống hoặc điều kiện **hiện tại** của môi trường tại một thời điểm, là cơ sở để agent đưa ra quyết định.

Action (Hành động)

Hành động là lựa chọn agent thực hiện, làm **thay đổi trạng thái** của hệ thống.

Policy (Chính sách)

Chính sách là kế hoạch định hướng ra quyết định của agent, **ánh xạ** trạng thái sang hành động.

Value Function (Hàm giá trị)

Hàm giá trị ước lượng **phần thưởng tích lũy kỳ vọng** mà agent có thể đạt được từ một trạng thái cho trước khi theo một chính sách cụ thể.

Model (Mô hình)

Mô hình biểu diễn **động học** của môi trường, cho phép agent mô phỏng kết quả tiềm năng của các cặp hành động-trạng thái phục vụ **lập kế hoạch**.

Chiến lược khám phá-khai thác (Exploration-Exploitation)

Chiến lược khám phá-khai thác là cách tiếp cận ra quyết định **cân bằng** giữa:

- **Khám phá** hành động mới để học thêm;
- **Khai thác** hành động đã biết để nhận phần thưởng ngay lập tức.

Learning Algorithm (Thuật toán học)

Thuật toán học là phương pháp agent **cập nhật** hàm giá trị hoặc chính sách dựa trên kinh nghiệm thu được khi tương tác với môi trường.

Experience Replay (Bộ nhớ kinh nghiệm)

Experience replay là kỹ thuật **lấy mẫu ngẫu nhiên** từ các kinh nghiệm đã lưu trước đó trong quá trình huấn luyện, nhằm tăng **ổn định** học và giảm tương quan giữa các sự kiện liên tiếp.

1.3 Cách hoạt động của Deep RL

Mạng nơ-ron và học thử-sai

Học tăng cường sâu dùng **mạng nơ-ron nhân tạo**, gồm các lớp nút mô phỏng hoạt động của nơ-ron trong não người. Các nút xử lý và truyền thông tin qua phương pháp **thử-sai** để xác định kết quả hiệu quả.

Chính sách, tìm kiếm và đại diện

Trong Deep RL, thuật ngữ **policy** chỉ chiến lược máy tính hình thành dựa trên phản hồi nhận được khi tương tác môi trường. Các policy này giúp máy tính ra quyết định bằng cách xét **trạng thái hiện tại** và **tập hành động** (các lựa chọn khả dụng). Khi chọn các lựa chọn đó, diễn ra quá trình gọi là **search** (tìm kiếm): máy tính đánh giá hành động khác nhau và quan sát kết quả. Khả năng phối hợp **học**, **ra quyết định** và **biểu diễn** có thể mang lại hiểu biết mới về cách não người vận hành.

Kiến trúc đặc trưng của Deep RL

Điểm tách biệt học tăng cường sâu nằm ở **kiến trúc**, cho phép học tương tự não người. Kiến trúc gồm **nhiều lớp** mạng nơ-ron đủ mạnh để xử lý dữ liệu **không nhãn** và **không có cấu trúc**.

1.4 Danh sách thuật toán Deep RL

Các thuật toán quan trọng

Dưới đây là một số thuật toán quan trọng trong học tăng cường sâu:

1. **Deep Q-Network** hoặc **Deep Q-Learning**
2. **Double Deep Q-Learning**
3. **Actor-Critic Method**
4. **Deep Deterministic Policy Gradient (DDPG)**

1.5 Ứng dụng của Deep RL

Các lĩnh vực ứng dụng nổi bật

Học tăng cường sâu được dùng rộng rãi trong nhiều lĩnh vực sau.

1.5.1 Trò chơi (Gaming)

Trò chơi điện tử

Deep RL được dùng phát triển trò chơi vượt xa khả năng con người thông thường. Các trò chơi thiết kế bằng Deep RL gồm game Atari 2600, cờ **Go**, **Poker** và nhiều thể loại khác.

1.5.2 Điều khiển robot (Robot Control)

Học tăng cường đối kháng mạnh

Lĩnh vực này dùng **học tăng cường đối kháng mạnh** (robust adversarial reinforcement learning): agent học vận hành khi có đối thủ gây **nhiều** lên hệ thống. Mục tiêu là phát triển chiến lược tối ưu để xử lý gián đoạn.

Robot trí tuệ nhân tạo

Robot do AI điều khiển có ứng dụng rộng: sản xuất, tự động hóa chuỗi cung ứng, y tế và nhiều lĩnh vực khác.

1.5.3 Xe tự lái (Self-driving Cars)

Lái tự động

Học tăng cường sâu là một trong các khái niệm then chốt của **lái tự động**. Kịch bản lái tự động đòi hỏi hiểu môi trường, tác nhân tương tác, đàm phán và ra quyết định **động**—điều chỉ có được nhờ học tăng cường.

1.5.4 Y tế (Healthcare)

Chăm sóc sức khỏe cá nhân hóa

Deep RL đã thúc đẩy nhiều tiến bộ trong y tế, ví dụ **cá nhân hóa** liều thuốc để tối ưu chăm sóc bệnh nhân, đặc biệt với bệnh **mãn tính**.

1.6 So sánh RL và Deep RL

Bảng so sánh Reinforcement Learning và Deep Reinforcement Learning

Tiêu chí	Reinforcement (RL)	Deep Reinforcement Learning (Deep RL)
Định nghĩa	Nhánh con học máy dùng phương pháp thử-sai để ra quyết định.	Nhánh con của RL tích hợp học sâu cho quyết định phức tạp hơn.
Xấp xỉ hàm (Function Approximation)	Dùng phương pháp đơn giản như bảng tra (tabular) để ước lượng giá trị.	Dùng mạng nơ-ron ước lượng giá trị, cho phép biểu diễn phức tạp hơn.
Biểu diễn trạng thái	Phụ thuộc đặc trưng thiết kế thủ công để mô tả môi trường.	Tự học đặc trưng liên quan từ dữ liệu đầu vào thô.
Độ phức tạp	Hiệu quả với môi trường đơn giản , không gian trạng thái/hành động nhỏ.	Hiệu quả với môi trường chiều cao , phức tạp.
Hiệu năng	Tốt ở môi trường đơn giản; khó với không gian lớn hoặc liên tục .	Vượt trội ở tác vụ phức tạp: game video, điều khiển robot.
Ứng dụng	Tác vụ cơ bản như game đơn giản.	Ứng dụng nâng cao: lái tự động, chơi game, điều khiển robot.

2 Thuật toán học tăng cường sâu (Deep RL Algorithms)

Thuật toán Deep RL

Thuật toán học tăng cường sâu là loại thuật toán học máy **kết hợp học sâu và học tăng cường**. Deep RL giải quyết bài toán cho agent tính toán học ra quyết định bằng cách tích hợp học sâu từ dữ liệu đầu vào không có cấu trúc, **không cần** kỹ thuật thủ công thiết kế không gian trạng thái. Các thuật toán này có khả năng quyết định hành động tối ưu mục tiêu ngay cả khi đầu vào có kích thước lớn.

2.1 Học tăng cường (Reinforcement Learning)

Vai trò của RL trong Deep RL

Học tăng cường gồm một agent học từ **phản hồi** đối với hành động của mình khi khám phá môi trường. Mục tiêu chính của agent là **tối đa hóa phần thưởng tích lũy** bằng cách phát triển chiến lược định hướng ra quyết định trong mọi tình huống có thể.

2.2 Vai trò của học sâu trong RL

Hạn chế xấp xỉ truyền thống

Trong thuật toán RL truyền thống, **bảng tra** hoặc xấp xỉ hàm cơ bản thường được dùng để biểu diễn hàm giá trị, chính sách hoặc mô hình. Các cách này **không đủ hiệu quả** trong bối cảnh khó như game video, robot hoặc xử lý ngôn ngữ tự nhiên.

Nền tảng Deep RL

Mạng nơ-ron cho phép xấp xỉ các hàm **phức tạp, đa chiều** thông qua học sâu. Đây là nền tảng của học tăng cường sâu.

2.3 Lợi ích kết hợp học sâu và RL

Ba lợi ích chính

Kết hợp mạng học sâu với học tăng cường mang lại các lợi ích sau:

1. Xử lý đầu vào **chiều cao** (ảnh thô, dữ liệu cảm biến liên tục).
2. Hiểu quan hệ **phức tạp** giữa trạng thái và hành động thông qua học.
3. Học **biểu diễn chung** bằng cách khái quát hóa giữa các trạng thái và hành động khác nhau.

2.4 Các thuật toán Deep RL phổ biến

Tổng quan

Dưới đây là một số thuật toán học tăng cường sâu phổ biến.

2.4.1 Deep Q-Networks (DQN)

Deep Q-Network

Deep Q-Network (DQN) là mở rộng của Q-learning cổ điển, dùng **mạng nơ-ron sâu** ước lượng hàm giá trị hành động $Q(s, a)$. Thay vì lưu giá trị Q trong bảng, DQN dùng mạng nơ-ron xử lý miền đầu vào phức tạp như **pixel game**. Nhờ đó RL có thể giải quyết tác vụ phức tạp—ví dụ chơi Atari, agent học từ đầu vào **trực quan**.

Ổn định huấn luyện DQN

DQN cải thiện ổn định huấn luyện qua hai phương pháp chính:

- **Experience replay**: lưu và lấy mẫu kinh nghiệm quá khứ;
- **Target network**: duy trì mục tiêu Q nhất quán bằng cách cập nhật định kỳ một mạng riêng.

Các cải tiến này giúp DQN học hiệu quả trong bối cảnh **quy mô lớn**.

2.4.2 Double Deep Q-Networks (DDQN)

Giảm thiên lệch overestimation

Double Deep Q-Network (DDQN) cải thiện DQN bằng cách giảm vấn đề **thiên lệch overestimation** khi cập nhật giá trị Q . Trong DQN thông thường, một mạng Q duy nhất vừa **chọn hành động** vừa **ước lượng giá trị**, dễ dẫn tới xấp xỉ giá trị **lạc quan quá mức**.

Hai mạng trong DDQN

DDQN dùng **hai mạng riêng**:

- Mạng Q **hiện tại** để chọn hành động;
- Mạng **target** để đánh giá hành động đó.

Việc tách vai trò giảm thiên lệch, nâng **độ chính xác học**. DDQN vẫn kế thừa experience replay và target network từ DQN để tăng **độ bền** và **tin cậy**.

2.4.3 Dueling Deep Q-Networks (Dueling DQN)

Dueling DQN

Dueling Deep Q-Networks mở rộng DQN chuẩn bằng cách **tách** giá trị Q thành hai thành phần:

- Hàm giá trị trạng thái $V(s)$;
- Hàm **advantage** $A(s, a)$, ước lượng mức giá trị của từng hành động so với giá trị trung bình.

Giá trị Q cuối cùng được ước lượng bằng cách **kết hợp** hai thành phần này.

Lợi ích biểu diễn tách rời

Biểu diễn này tăng **sức mạnh và hiệu quả** của Q-learning: mô hình ước lượng $V(s)$ chính xác hơn, đồng thời giảm nhu cầu ước lượng giá trị hành động chi tiết trong một số tình huống.

2.4.4 Policy Gradient Methods

Phương pháp Policy Gradient

Policy Gradient là nhóm thuật toán theo hướng **lập chính sách**: điều chỉnh trực tiếp policy để đạt policy tối ưu **tối đa hóa phần thưởng kỳ vọng**. Thay vì chỉ học hàm giá trị, các phương pháp này **tối ưu policy** theo gradient của mục tiêu định nghĩa đối với tham số policy.

Mục tiêu và các biến thể

Mục tiêu chính là tính **gradient phần thưởng trung bình** và cập nhật chiến lược. Các thuật toán tiêu biểu: **REINFORCE**, **Actor-Critic**, **Proximal Policy Optimization (PPO)**. Các cách tiếp cận này hiệu quả trong không gian hành động **chiều cao** hoặc **liên tục**.

2.4.5 Proximal Policy Optimization (PPO)

PPO

Proximal Policy Optimization (PPO) là thuật toán RL nhằm tối ưu chính sách **ổn định và hiệu quả** hơn. PPO cập nhật policy bằng cách tối đa hóa hàm mục tiêu gắn với policy, nhưng **giới hạn** mức thay đổi policy cho phép để tránh biến động đột ngột.

Mục tiêu clipped

Policy mới **không được** lệch quá xa policy cũ; PPO dùng **mục tiêu clipped** để đảm bảo điều đó. Nhờ mục tiêu clipped, PPO ngăn thay đổi policy quá lớn giữa hai lần cập nhật liên tiếp. Sự cân bằng giữa khám phá và khai thác giúp tránh **suy giảm hiệu năng**

và thúc đẩy **hội tụ** **mượt** hơn.

Phạm vi ứng dụng PPO

PPO được áp dụng trong Deep RL cho cả không gian hành động **liên tục** và **rời rạc** nhờ tính **đơn giản** và **hiệu quả**.

3 Mạng Q sâu (Deep Q-Networks)

3.1 Deep Q-Network là gì?

Deep Q-Network (DQN)

Deep Q-Network (DQN) là thuật toán trong học tăng cường, kết hợp **mạng nơ-ron sâu** với **Q-learning**, giúp agent học policy tối ưu trong môi trường phức tạp. Q-learning truyền thống hiệu quả khi số trạng thái **nhỏ và hữu hạn**, nhưng gặp khó với không gian trạng thái **lớn** hoặc **liên tục** vì kích thước bảng Q . DQN khắc phục bằng cách thay bảng Q bằng mạng nơ-ron **xấp xỉ** giá trị Q cho mọi cặp trạng thái–hành động.

3.2 Các thành phần chính

Kiến trúc DQN

Dưới đây là các thành phần trong kiến trúc Deep Q-Networks.

Lớp đầu vào (Input Layer)

Lớp đầu vào nhận thông tin trạng thái từ môi trường dưới dạng **vector giá trị số**.

Lớp ẩn (Hidden Layers)

Lớp ẩn của DQN gồm nhiều nơ-ron **fully connected** biến đổi dữ liệu đầu vào thành đặc trưng phức tạp hơn, phù hợp cho **dự đoán**.

Lớp đầu ra (Output Layer)

Lớp đầu ra: mỗi hành động khả dĩ ở trạng thái hiện tại tương ứng **một nơ-ron**. Giá trị đầu ra của các nơ-ron này là **ước lượng giá trị** của từng hành động trong trạng thái đó.

Bộ nhớ (Memory)

Bộ nhớ: DQN dùng **memory replay** lưu các sự kiện huấn luyện của agent. Mọi thông tin—trạng thái hiện tại, hành động đã chọn, phần thưởng nhận được, trạng thái kế tiếp—được lưu dưới dạng **bộ** (tuple) trong bộ nhớ.

Hàm mất mát (Loss Function)

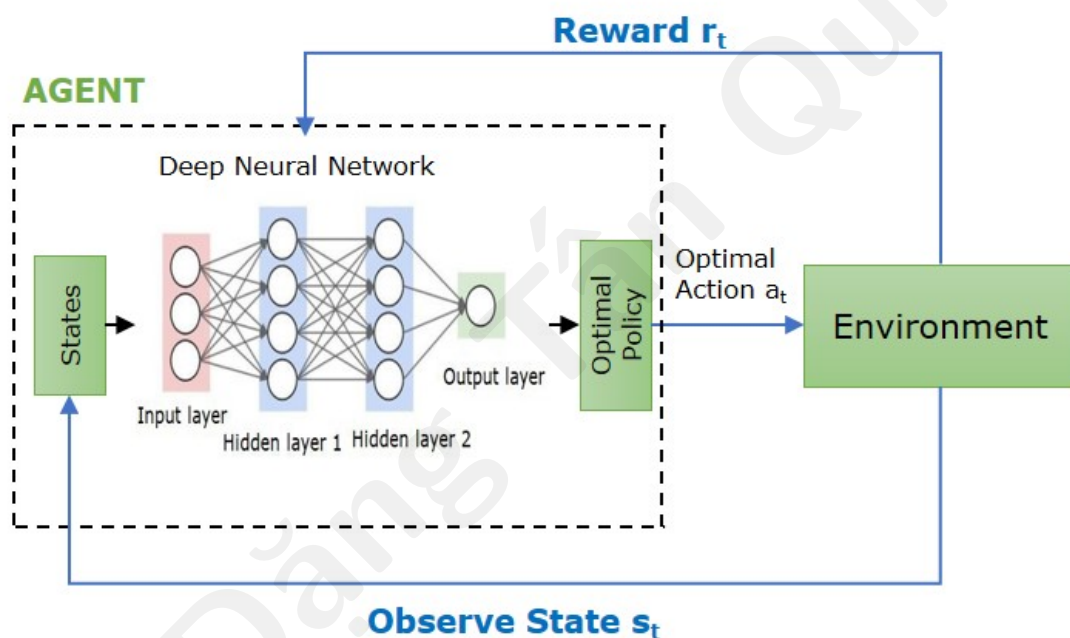
Hàm mất mát: DQN tính **chênh lệch** giữa giá trị Q thực tế từ replay memory và giá trị Q dự đoán để xác định loss.

Tối ưu hóa (Optimization)

Tối ưu hóa: điều chỉnh trọng số mạng để **tối thiểu hóa** hàm mất mát. Thường dùng **stochastic gradient descent (SGD)** cho mục đích này.

Sơ đồ kiến trúc DQN

Hình sau minh họa các thành phần trong kiến trúc Deep Q-Network:

**3.3 Cách hoạt động của DQN****Các bước vận hành**

Quy trình làm việc của DQN gồm các bước sau.

3.3.1 Kiến trúc mạng nơ-ron

Neural Network Architecture

DQN dùng chuỗi **frame** (ví dụ ảnh từ game) làm đầu vào và sinh tập giá trị Q cho mọi hành động khả dĩ ở trạng thái đó. Cấu hình điển hình gồm lớp **convolutional** nắm quan hệ không gian và lớp **fully connected** xuất ra giá trị Q .

3.3.2 Experience Replay

Bộ đệm kinh nghiệm

Trong huấn luyện, agent lưu các tương tác (s, a, r, s') vào **replay buffer**. Lấy mẫu **ngẫu nhiên** các batch từ buffer để huấn luyện mạng, giảm tương quan giữa kinh nghiệm liên tiếp và tăng **ổn định** huấn luyện.

3.3.3 Target Network

Mạng mục tiêu

Để ổn định quá trình huấn luyện, DQN dùng **mạng target riêng** sinh mục tiêu giá trị Q . Mạng target được **cập nhật định kỳ** trọng số từ mạng chính để giảm rủi ro **phân kỳ** khi huấn luyện.

3.3.4 Chính sách Epsilon-Greedy

Epsilon-Greedy Policy

Agent dùng chiến lược **epsilon-greedy**:

- Với xác suất ε : chọn hành động **ngẫu nhiên**;
- Với xác suất $1 - \varepsilon$: chọn hành động có giá trị Q **cao nhất**.

Cân bằng khám phá-khai thác giúp agent học hiệu quả.

3.3.5 Quy trình huấn luyện

Training Process

Mạng nơ-ron được huấn luyện bằng **gradient descent** để tối thiểu hóa loss giữa Q dự đoán và Q mục tiêu. Giá trị Q mục tiêu tính theo **phương trình Bellman**, kết hợp phần thưởng nhận được và giá trị Q **lớn nhất** của trạng thái kế tiếp.

3.4 Hạn chế của Deep Q-Networks

Các hạn chế chính

Deep Q-Networks có một số hạn chế ảnh hưởng hiệu quả và hiệu năng:

1. **Bất ổn định** do vấn đề non-stationarity từ việc cập nhật mạng nơ-ron thường xuyên.
2. Đôi khi **overestimate** giá trị Q , gây tác động tiêu cực lên quá trình học.
3. Cần **nhiều mẫu** để học tốt—tốn kém và tốn thời gian tính toán.
4. Hiệu năng phụ thuộc mạnh vào **siêu tham số**: learning rate, hệ số chiết khấu, tỷ lệ khám phá.
5. DQN chủ yếu cho không gian hành động **rời rạc**; khó với hành động **liên tục**.

3.5 Double Deep Q-Networks

Double DQN

Double DQN là phiên bản mở rộng của DQN, nhằm xử lý vấn đề **thiên lệch overestimation** trong cập nhật giá trị Q . Thiên lệch này xuất phát từ việc quy tắc cập nhật Q-learning dùng **cùng một** mạng Q để vừa chọn vừa đánh giá hành động, dẫn tới ước lượng Q **phồng lên**. Điều này gây **bất ổn** huấn luyện và cản trở học.

Hai mạng tách vai trò

Double DQN dùng hai mạng:

- **Q-Network**: chọn hành động;
- **Target Network**: đánh giá **giá trị** của hành động đã chọn.

Cách tính mục tiêu

Thay đổi chính nằm ở cách tính **target**: thay vì dùng một mạng Q cho cả chọn và đánh giá hành động kế tiếp, Double DQN dùng mạng Q để **chọn** hành động ở trạng thái sau và mạng target để **đánh giá** giá trị Q của hành động đó. Việc tách này giảm xu hướng overestimate, cho ước lượng giá trị **chính xác** hơn.

Ổn định hơn DQN chuẩn

Double DQN mang lại quá trình huấn luyện **nhất quán và đáng tin cậy** hơn, đặc biệt trong kịch bản như game Atari, nơi DQN thông thường có thể gặp khó với overestimation.

3.6 Dueling Deep Q-Networks

Dueling DQN

Dueling Deep Q-Networks cải thiện quá trình học của DQN truyền thống bằng cách tách ước lượng giá trị trạng thái khỏi **advantage** hành động. Trong DQN cổ điển, mỗi cặp trạng thái–hành động có một giá trị Q riêng biểu diễn phần thưởng tích lũy kỳ vọng. Cách này có thể **kém hiệu quả** khi nhiều hành động dẫn tới hậu quả tương tự.

Công thức phân rã Q

Dueling DQN phân rã giá trị Q thành hai phần:

- **Giá trị trạng thái** $V(s)$: giá trị của việc ở trạng thái s ;
- **Hàm advantage** $A(s, a)$: mức tốt hơn của hành động a so với các hành động khác trong cùng trạng thái.

Giá trị Q được cho bởi:

$$Q(s, a) = V(s) + A(s, a).$$

Lợi ích học tách rời

Dueling DQN giúp agent hiểu môi trường sâu hơn và tránh học các ước lượng giá trị hành động **không cần thiết**. Hiệu năng cải thiện, đặc biệt khi phần thưởng **trễ**, vì agent nắm rõ hơn tầm quan trọng của các trạng thái khi chọn hành động tối ưu.

4 Deep Deterministic Policy Gradient (DDPG)

DDPG

Deep Deterministic Policy Gradient (DDPG) là thuật toán học tăng cường đồng thời học **hàm Q** và **chính sách (policy)**. Hàm Q được học từ dữ liệu **off-policy** và phương trình Bellman; sau đó dùng để cập nhật policy. DDPG mở rộng **Deep Q-Networks (DQN)** sang **không gian hành động liên tục**, với policy **xác định (deterministic)** thay vì policy ngẫu nhiên như trong DQN hay REINFORCE.

Bối cảnh và đặc điểm

DDPG được thiết kế cho bài toán có **hành động liên tục** (điều khiển robot, lái xe tự động, điều khiển động cơ...). Thuật toán **model-free**, **off-policy**, dựa trên kiến trúc **actor–critic**, kết hợp ý tưởng **Q -learning** (học giá trị) và **policy gradient** (học chính sách). Deep learning dùng để xấp xỉ cả hàm giá trị lẫn policy khi quan sát và hành động có chiều cao.

4.1 Khái niệm then chốt

Định lý Policy Gradient (xác định)

DDPG dùng **định lý gradient chính sách xác định** (deterministic policy gradient theorem): gradient của **return kỳ vọng** theo tham số policy $\mu_\theta(s)$ có thể tính qua đạo hàm của $Q(s, a)$ theo hành động a tại $a = \mu_\theta(s)$. Gradient này dùng để cập nhật mạng **actor**.

Off-policy

DDPG là **off-policy**: học từ kinh nghiệm do **chính sách hành vi (behavior policy)** tạo ra — có thể khác policy đang tối ưu — nhờ lưu các bước (s, a, r, s') vào **replay buffer** rồi lấy mẫu mini-batch ngẫu nhiên để cập nhật mạng.

“Deterministic” trong DDPG

Chính sách xác định ánh xạ mỗi trạng thái s sang **một** hành động cụ thể $a = \mu_\theta(s)$, không phải phân phối xác suất $\pi(a|s)$ như REINFORCE hay policy gradient ngẫu nhiên. So với hàm giá trị/hàm Q (thường gán điểm số cho mọi hành động khả dĩ), actor chỉ **xuất ra một vector hành động** cho mỗi s . Phù hợp môi trường liên tục nơi hành động là số thực (vận tốc, góc lái...).

4.2 Thành phần cốt lõi (actor–critic)

Kiến trúc Actor–Critic

- **Actor** (mạng policy): nhận s , xuất hành động xác định $a = \mu_\theta(s)$; tham số θ .
- **Critic** (mạng Q): xấp xỉ $Q_\phi(s, a)$ — giá trị kỳ vọng return khi ở s và thực hiện a ; nhận cả s và a ; tham số ϕ .

Actor quyết định **hành động**; critic **chấm điểm** hành động đó để hướng cập nhật actor.

Policy xác định so với policy ngẫu nhiên

Với mỗi s , actor trả về **một** a , không lấy mẫu từ phân phối. Điều này giúp gradient theo hành động liên tục ổn định hơn, nhưng **khám phá** phải bổ sung bằng nhiều (xem mục Ornstein–Uhlenbeck).

4.3 Cách DDPG hoạt động

Không gian hành động liên tục

Khác môi trường rời rạc (game với tập hành động hữu hạn), DDPG xử lý $a \in \mathbb{R}^d$ (tốc độ, moment, torque...). Critic học $Q(s, a)$ trên toàn miền liên tục; actor tối đa hóa Q theo a cho mỗi s .

Experience replay

Lưu kinh nghiệm dạng bộ tứ:

$$(s_t, a_t, r_t, s_{t+1})$$

vào buffer có dung lượng cố định. Mỗi bước huấn luyện **lấy ngẫu nhiên** mini-batch từ buffer thay vì chỉ dùng dữ liệu liên tiếp theo thời gian. Điều này:

- giảm **tương quan** giữa các mẫu liên kề;
- tái sử dụng dữ liệu cũ (hiệu quả mẫu cho off-policy);
- ổn định hơn khi cập nhật mạng sâu.

Huấn luyện Critic (TD)

Critic cập nhật theo **Temporal Difference**, thường dạng $TD(0)$. Mục tiêu xấp xỉ Bellman với policy **target** $\mu_{\theta'}$:

$$y = r + \gamma Q_{\phi'}(s', \mu_{\theta'}(s'))$$

Giảm sai số TD (MSE giữa $Q_{\phi}(s, a)$ và y), ví dụ:

$$L_{\text{critic}} = \mathbb{E}[(Q_{\phi}(s, a) - y)^2]$$

Critic đánh giá chất lượng quyết định của actor qua Q .

Huấn luyện Actor (gradient ascent trên Q)

Actor cập nhật để **tăng** $Q(s, \mu_{\theta}(s))$: tính $\nabla_{\theta} Q_{\phi}(s, \mu_{\theta}(s))$ (chuỗi quy tắc qua hành động do actor sinh ra), rồi đi theo hướng **gradient ascent** trên θ — ưu tiên hành động mang Q cao hơn. Đây là bước “policy improvement” trong vòng actor-critic.

4.4 Mạng target và soft update**Mạng target**

DDPG duy trì bản sao **chậm** của actor và critic: $\mu_{\theta'}$, $Q_{\phi'}$. Target dùng trong y ở bước critic để tránh “đuổi theo” mục tiêu thay đổi quá nhanh — nguyên nhân hay gặp của **dao động** khi dùng xấp xỉ hàm sâu (tương tự DQN).

Soft update (Polyak averaging)

Thay vì copy cứng toàn bộ trọng số mỗi C bước, DDPG **cập nhật mềm** target từng bước nhỏ:

$$\theta' \leftarrow \tau \theta + (1 - \tau) \theta', \quad \phi' \leftarrow \tau \phi + (1 - \tau) \phi'$$

với $\tau \ll 1$ (ví dụ 10^{-3}) để target thay đổi **chậm**, tăng ổn định.

Công thức soft update

Một số tài liệu ghi nhầm dạng $\theta' \leftarrow \tau + (1 - \tau)\theta'$ (thiếu θ ở vế phải). Công thức chuẩn trong bài gốc DDPG (Lillicrap et al.) và triển khai phổ biến là **trộn có trọng số** giữa mạng online và mạng target như trên, áp dụng riêng cho actor (θ) và critic (ϕ).

4.5 Khám phá–khai thác**Nhiều Ornstein–Uhlenbeck (OU)**

Policy xác định vốn **tham lam**: luôn chọn hành động “tốt nhất” theo mạng hiện tại, ít khám phá. DDPG thêm **nhiều OU** có tương quan theo thời gian vào hành động thực thi trong môi trường:

$$a_{\text{exec}} = \mu_{\theta}(s) + \mathcal{N}_{\text{OU}}$$

OU mô phỏng quán tính (phù hợp điều khiển liên tục) hơn nhiều Gaussian trắng từng bước. Khi thu thập vào replay buffer vẫn lưu hành động đã thực hiện (có nhiều); khi học, actor vẫn hướng tới tối đa Q của policy xác định.

Exploration vs. exploitation

Giai đoạn đầu cần nhiều đủ lớn; về sau có thể giảm biên độ nhiễu. Nếu chỉ khai thác policy hiện tại, agent dễ kẹt ở cực tiểu cục bộ — đặc biệt trên không gian hành động liên tục, phức tạp.

4.6 Thách thức**Thách thức chính của DDPG**

- **Không ổn định khi huấn luyện**: xấp xỉ hàm bằng mạng sâu dễ dao động; target network và replay buffer giúp nhưng vẫn **nhạy hyperparameter** (tốc độ học actor/critic, τ , kích thước batch, kiến trúc).
- **Khám phá kém**: dù có nhiều OU, môi trường rất phức tạp hoặc reward thưa vẫn có thể học chậm hoặc hội tụ kém; cần điều chỉnh nhiễu, schedule, hoặc thuật toán kế thừa (TD3, SAC).
- **Overestimation / bias Q**: critic lạc kéo actor sai; các biến thể như **Twin Delayed DDPG (TD3)** giảm vấn đề này.
- **Hyperparameter và tái lập**: kết quả phụ thuộc seed, scale reward, chuẩn hóa quan sát — cần thực hành tinh chỉnh cẩn thận.

Tóm tắt luồng DDPG

1. Agent tương tác môi trường với $a = \mu_{\theta}(s) + \text{nhiều}$, lưu (s, a, r, s') vào replay.
2. Lấy mini-batch; cập nhật critic bằng TD target từ $Q_{\phi'}$, $\mu_{\theta'}$.
3. Cập nhật actor theo gradient của $Q_{\phi}(s, \mu_{\theta}(s))$ theo θ .

4. Soft-update θ', ϕ' ; lặp.

5 Phương pháp vùng tin cậy (Trust Region)

Vai trò trong học tăng cường sâu

Trong **tối ưu chính sách (policy optimization)**, mục tiêu là cải thiện policy để tăng phần thưởng kỳ vọng mà **không làm hành vi agent sụp đổ** sau một bước cập nhật quá lớn. Với mạng nơ-ron sâu làm xấp xỉ policy, gradient một bước có thể đẩy tham số xa khỏi vùng mà xấp xỉ còn đáng tin — gây **dao động**, sụt hiệu năng, hoặc hội tụ kém. **Vùng tin cậy (trust region)** giới hạn mức thay đổi policy mỗi lần cập nhật để huấn luyện **mượt** và **đáng tin** hơn.

5.1 Định nghĩa vùng tin cậy

Trust region

Vùng tin cậy là khái niệm tối ưu: trong mỗi bước lặp, chỉ cho phép cập nhật tham số (policy, hoặc hàm giá trị) nằm trong **tập tin cậy** quanh điểm hiện tại — nơi mô hình xấp xỉ (surrogate) hoặc gradient còn phản ánh đúng hành vi thật. Ngoài vùng đó, bước cập nhật bị coi là **không đáng tin** và bị cắt hoặc chiếu lại.

Hệ quả khi huấn luyện policy

- Giới hạn **độ lớn** thay đổi tham số policy mỗi iteration.
- Tránh thay đổi **đột ngột** làm phân phối hành động lệch xa dữ liệu đã thu thập.
- Cân bằng giữa **cải thiện mục tiêu** và **ổn định** học.

5.2 Vai trò trong tối ưu chính sách

Policy gradient và trust region

Policy gradient tối đa hóa $J(\theta) = \mathbb{E}[G]$ bằng cách leo theo $\nabla_{\theta} J$. Không có ràng buộc, một bước SGD/Adam có thể quá lớn — đặc biệt khi dùng **function approximator** sâu — dẫn tới policy mới kém hẳn dù gradient cục bộ dương. Trust region **ép** mỗi bước chỉ “đi xa” một mức được phép trong metric phù hợp (thường liên quan phân phối hành động).

KL divergence

Kullback–Leibler (KL) divergence $D_{\text{KL}}(\pi_{\theta_{\text{old}}} \parallel \pi_{\theta})$ đo mức **lệch** giữa policy cũ và mới. TRPO dùng ràng buộc dạng $D_{\text{KL}} \leq \delta$: thay đổi **nhỏ** theo KL thường cải thiện hiệu năng **ổn định**; thay đổi **lớn** dễ phá vỡ xấp xỉ và gây sụt reward.

Surrogate objective và clipping (PPO)

Hàm surrogate (mục tiêu thay thế) xấp xỉ lợi ích policy mới bằng tỷ số xác suất $\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}$ nhân advantage — đúng khi policy mới gần policy cũ. **PPO** ước lượng trust region qua **clipping**: cắt tỷ số trong $[1 - \epsilon, 1 + \epsilon]$ để phạt cập nhật quá mạnh, không cần giải tối ưu ràng buộc KL đầy đủ như TRPO.

5.3 TRPO — Trust Region Policy Optimization**TRPO**

Trust Region Policy Optimization (TRPO) tối đa hóa **surrogate objective** (thường dạng advantage-weighted ratio) dưới **ràng buộc** trust region bằng KL: mỗi bước tìm cập nhật policy cải thiện mục tiêu nhưng $D_{\text{KL}}(\pi_{\text{old}} \parallel \pi_{\text{new}}) \leq \delta$. Mục đích: khắc phục bước policy gradient **quá lớn, không ổn định** khi dùng mạng sâu.

Ý tưởng thuật toán TRPO

1. Thu thập rollout với policy hiện tại $\pi_{\theta_{\text{old}}}$.
2. Ước lượng **advantage** $A(s, a)$ (GAE, value baseline...).
3. Tối đa hóa surrogate có kỳ vọng $\mathbb{E}\left[\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A(s, a)\right]$ với ràng buộc $\text{KL} \leq \delta$.
4. Giải bài toán con bằng **natural gradient** / xấp xỉ Fisher information (thực tế dùng conjugate gradient, line search).
5. Gán $\theta_{\text{old}} \leftarrow \theta$; lặp.

Ưu và nhược TRPO

- **Ưu**: cập nhật policy **ổn định**, lý thuyết rõ về monotonic improvement (trong điều kiện xấp xỉ đúng).
- **Nhược**: triển khai **phức tạp**, tính toán KL và Fisher tốn kém; khó scale như PPO trên GPU thuần mini-batch.

5.4 PPO — Proximal Policy Optimization**PPO**

Proximal Policy Optimization (PPO) nhằm **đơn giản hóa** trust region: dùng cùng ý tưởng surrogate nhưng **giới hạn bước** bằng **clipped objective** thay vì ràng buộc KL cứng. Policy mới không được lệch quá xa policy cũ (theo tỷ số xác suất), vẫn cân bằng khám phá–khai thác trong on-policy rollout.

Mục tiêu clipped (PPO-clip)

Với $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ và advantage $\hat{A}_t$:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

Cập nhật θ bằng vài epoch **stochastic gradient ascent** trên cùng batch rollout (có thể kèm value loss, entropy bonus).

Quy trình PPO điển hình

1. Thu thập tập kinh nghiệm on-policy với $\pi_{\theta_{\text{old}}}$.
2. Tính return và advantage (thường GAE).
3. Tối ưu L^{CLIP} (và loss critic) nhiều mini-batch epoch trên cùng dữ liệu.
4. Đồng bộ θ_{old} ; lặp thu thập mới.

Vì sao PPO phổ biến

Đơn giản hơn TRPO, dễ song song hóa, **ổn định** trên robot, game, điều khiển; ít phụ thuộc giải tối ưu con phức tạp. Là lựa chọn mặc định cho nhiều bài toán **policy gradient on-policy** sâu.

5.5 Natural Gradient Descent**Natural gradient**

Natural gradient descent điều chỉnh hướng và độ dài bước theo **độ cong** của mặt phần thưởng trong không gian phân phối (metric Fisher). Một bước natural gradient tương đương di chuyển trong **trust region** quanh policy hiện tại — hiệu quả khi không gian tham số **chiều cao**.

Liên hệ TRPO và natural gradient

TRPO xấp xỉ giải bài toán constrained bằng **natural gradient** kèm line search để thỏa $KL \leq \delta$. PPO thay ràng buộc KL bằng **clipping** thực dụng hơn nhưng cùng triết lý: không bước policy quá xa trong một lần cập nhật.

5.6 So sánh nhanh

TRPO, PPO và natural gradient

Tiêu chí	TRPO	PPO	Natural gradient
Cơ chế trust region	Ràng buộc KL cứng ($\leq \delta$).	Clipping surrogate / PPO-penalty.	Metric Fisher, bước trong vùng tin cậy.
Độ phức tạp	Cao (CG, line search).	Thấp hơn, SGD mini-batch.	Trung bình–cao (ước lượng Fisher).
On-policy	Có.	Có.	Thường dùng trong khung on-policy.
Ứng dụng	Nền tảng lý thuyết; ít dùng trực tiếp hơn PPO.	Robot, Atari, LLM RLHF...	Lỗi của TRPO; có thể độc lập.

5.7 Thách thức**Thách thức khi dùng trust region**

- **Xấp xỉ và vi phạm ràng buộc:** TRPO/PPO dựa trên surrogate và ước lượng advantage; xấp xỉ sai có thể **vi phạm** KL hoặc clipping không đủ chặt — policy vẫn sụt.
- **Chi phí tính toán:** TRPO và natural gradient (Fisher, KL) tốn hơn PPO thuần; không gian tham số lớn càng nặng.
- **Nhu cầu mẫu:** phương pháp on-policy trust region cần **nhiều rollout** mới mỗi vòng; sample efficiency thấp hơn off-policy (SAC, DDPG).
- **Hyperparameter:** δ (KL), ϵ (clip), số epoch PPO, coefficient entropy/value — **nhạy**, cần kinh nghiệm tinh chỉnh.
- **Hành động liên tục:** PPO/TRPO chuẩn cho policy rời rạc hoặc Gaussian; bài toán liên tục phức tạp thường chuyển sang SAC/TD3 thay vì trust region thuần.

Thuật toán deep RL dùng trust region

- **TRPO:** ràng buộc KL, surrogate constrained — nền tảng lý thuyết trust region trong RL sâu.
- **PPO:** clipping (hoặc penalty KL) — chuẩn công nghiệp on-policy ổn định.
- **Natural gradient:** điều chỉnh bước theo geometry policy; lỗi của TRPO, có thể kết hợp các mục tiêu khác.